

HW#5 : uC/OS-III USART LED Command Control

202055575 이동근

넋두리

과제를 시작하기 앞서, 필자는 본디 windows on ARM 노트북위에 WSL을 올리고 usbipd로 포트를 바인딩시켜 stm32를 빌드하고 플래시하는 환경을 구축했었다. 해당 방식으로도 잘 작동했으나 얼마전에 N100 프로세서가 탑재된 미니 PC를 구매했고 해당 PC를 집 밖에서도 원격으로 접속이 가능해서 노트북과 STM32 보드를 들고다니지 않아도 개발이 가능하게 되었다. N100의 운영체제는 우분투로 구성했고 WSL에서 한번 환경설정을 해본 경험이 있어서 그리 어렵지 않았다. N100에서 빌드환경을 구축했고, SSH로 데스크탑이나 랩탑에서 접속한뒤 개발가능한 환경을 구축했다.

이렇게 환경을 옮기고 나니 좋은 점이 한둘이 아니었다. 보드는 미니 PC 옆에 상시 연결해두고, 필자는 어느 기기에서든 SSH로 붙어서 빌드하고 플래시하면 그만이었다. WoA 환경에서 usbipd로 매번 usb를 attach 해주던 수고로움도 사라졌다. 우분투에서는 ST-LINK가 native하게 잡히니 드라이버 호환성 문제로 골치를 썩일 일도 없었다. 다만 한 가지, 처음 플래시를 시도할 때 OpenOCD가 ST-LINK에 접근하지 못하는 문제는 짚고 넘어가야 했다.

```
Error: libusb_open() failed with LIBUSB_ERROR_ACCESS
Error: open failed
```

이는 리눅스에서 일반 사용자가 ST-LINK USB 장치에 접근할 권한이 없어서 발생하는 문제다. udev rule을 추가하고 사용자를 적절한 그룹에 넣어준 뒤로는 sudo 없이도 OpenOCD가 보드를 정상적으로 물 수 있었다. 만약 필자처럼 리눅스 환경에서 ST-LINK가 권한 문제로 안 잡히는 학생이 있다면, IDE를 탭하기 전에 udev rule부터 확인하길 권한다.

개발 환경 및 빌드 확인

이 프로젝트는 HW#4에서 정리해둔 그대로 CMake와 ninja 기반으로 구성되어 있다. 실제 펌웨어 빌드는 ARM Cortex-M용 크로스 컴파일러인 `arm-none-eabi-gcc` 로 수행하고, `clangd` 는 VS Code에서 코드 분석과 자동완성 용도로만 쓰는 구조다. 일반 PC용 `gcc` 나 `clang` 으로는 baremetal STM32 펌웨어를 빌드할 수 없으니 이 구분은 꼭 필요하다.

빌드는 다음 한 줄이면 된다.

```
cmake --build build/ninja
```

플래시는 VS Code task의 `flash` target에 OpenOCD와 ST-LINK 설정을 묶어두었기에 task만 실행하면 알아서 올라간다. 시리얼 통신은 보드의 virtual COM port를 통하는데, 현재 환경에서는 `/dev/ttyACM0` 로 잡혔다. 안정적으로 쓰려면 `/dev/serial/by-id/...` 경로를 쓰는 편이 낫다. 시리얼 모니터는 115200 baud, 8 data bits, no parity, 1 stop bit로 맞췄다. USART3 초기화 코드에서도 동일하게 설정해두었다.

```

usart_init.USART_BaudRate = 115200;
usart_init.USART_WordLength = USART_WordLength_8b;
usart_init.USART_StopBits = USART_StopBits_1;
usart_init.USART_Parity = USART_Parity_No;

```

과제 목표

이번 HW#5는 제공받은 `app.c` starter code를 기반으로 STM32F429ZI 보드 위에서 uC/OS-III의 task를 활용해 USART 입력과 LED 제어를 구현하는 것이 목표였다. 과제는 두 단계로 나뉘었다.

5-1은 USART로 입력한 문자를 다시 시리얼 터미널에 되돌려주는 echo 기능과, 주기적으로 깜빡이는 LED blink task를 구현하는 것. 5-2는 USART로 문자열 명령을 입력받아 LED1, LED2, LED3를 각각 `on`, `off`, `blink` 상태로 제어하는 command 기능을 구현하는 것이다. 두 task는 반드시 별도로 생성하고, task 간 통신은 global variable로, 그 변수 접근은 critical section으로 보호하라는 것이 핵심 요구사항이었다.

5-1 : USART echo와 LED blink

가장 먼저 한 일은 task를 둘로 쪼개는 것이었다. 과제가 USART task와 LED task를 분리하라고 못박았으니, 하나의 task에서 모든 걸 처리하는 게으른 구현은 애초에 선택지에 없었다. `AppTaskCreate()` 에서 두 task를 각각 다른 우선순위로 생성했다.

```

OSTaskCreate(&UsartTaskTCB, "Usart Task", AppTaskUsart, 0u,
             APP_TASK_USART_PRIORITY, ...);

OSTaskCreate(&LedTaskTCB, "Led Task", AppTaskLed, 0u,
             APP_TASK_LED_PRIORITY, ...);

```

USART task의 우선순위(10)를 LED task(11)보다 높게 두었다. 입력 응답성이 더 중요하다고 판단했기 때문이다. 다만 두 task 모두 `OSTimeDlyHMSM` 으로 주기적으로 sleep 상태에 들어가기 때문에, 우선순위가 높은 USART task가 CPU를 독점해 LED task를 굶기는 일은 일어나지 않는다.

USART echo는 polling 방식으로 구현했다. ISR을 쓸 수도 있었지만 그 얘기는 뒤에서 따로 풀겠다. `USART_GetFlagStatus(USART3, USART_FLAG_RXNE)` 로 수신 플래그를 확인하고, 들어온 문자가 있을 때만 읽어서 그대로 다시 송신한다.

```

static char UsartGetChar(void)
{
    if (USART_GetFlagStatus(USART3, USART_FLAG_RXNE) == SET){
        char ch = (char)(USART_ReceiveData(USART3) & 0xFFu);
        return ch;
    }
    return '\0';
}

```

문자가 없으면 `'\0'` 을 돌려주도록 해서, task 루프에서는 `'\0'` 이 아닐 때만 echo와 명령 처리를 하도록 했다. 들어온 문자는 `UsartPutChar(ch)` 로 화면에 다시 찍어준다. 이렇게 하면 사용자가 친 글자가 시리얼 터미널에 보이게 된다.

LED blink는 LED task가 매 주기마다 LED 상태를 뒤집는 방식이다. 5-1 단계에서는 거창한 명령 처리 없이 LED가 일정 주기로 on/off를 반복하는지만 확인하면 되었다. 두 기능이 별도 task에서 도니, LED가 깜빡이는 와중에도 USART echo가 멈추지 않고, 반대로 글자를 입력하는 중에도 LED blink가 끊기지 않는다. 이것이 RTOS에서 task를 나누는 이유 그 자체다.

5-2 : LED command 구현

두 번째 소과제부터가 진짜였다. 요구된 명령 형식은 `"led" + 00 + 00` 꼴이고, 정리하면 다음과 같다.

명령 예시	동작
<code>led3on</code>	LED3 on
<code>led2off</code>	LED2 off
<code>led1blink3</code>	LED1을 3초 주기로 blink
<code>reset</code>	모든 LED off

명령을 받기 위해 USART task는 입력 문자를 하나씩 command buffer에 쌓아두고, 매 입력마다 지금까지 쌓인 문자열을 parser에 넘긴다.

```
CmdBuffer[CmdLen++] = ch;
CmdBuffer[CmdLen] = '\0';

UsartPutChar(ch);
HandleCommand(CmdBuffer);
```

처음에는 Enter를 쳤을 때 한 번에 명령을 처리하는 방식도 떠올렸으나, 최종적으로는 한 글자가 들어올 때마다 parser를 호출하는 방식을 택했다. 이렇게 하면 명령이 완성되는 즉시 반응하니 사용감이 더 즉각적이다. 대신 parser 입장에서는 골치 아픈 문제가 하나 생긴다. 아직 입력이 덜 끝난 상태와, 명백히 틀린 명령을 구분해야 한다는 점이다.

예컨대 `led10off` 를 치는 도중의 `led10` , `led10f` 는 아직 완성되지 않은 정상 입력이므로 곧바로 Wrong Command로 내치면 안 된다. 반면 `xyz` 같이 형식 자체가 어긋난 문자열은 즉시 buffer를 비워야 한다. 이 둘을 가르는 것이 parser 설계의 전부라 해도 과언이 아니었다.

문자열 비교와 명령 길이 처리

parser를 짜면서 `Str_Cmp_N(a, b, n)` 의 동작을 곱씹어봤다. 이 함수는 앞에서부터 `n` 글자만 비교하기 때문에, 비교 길이를 잘못 잡으면 prefix만 맞아도 같은 명령으로 오인할 수 있다. 그래서 필자는 "지금까지 입력된 길이만큼만 비교한다"는 발상을 썼다. `min()` 헬퍼를 직접 만들어 비교 길이를 현재 입력 길이로 잘라낸 것이다.

```
static int min(int a, int b){
    if(a>b) return b;
    else return a;
}
```

`reset` 을 판별하는 부분을 보자.

```

if (Str_Cmp_N(cmd, "reset", min(cmdlen,5)) == 0 ){
    if(cmdlen < 5 ) break;    // 아직 덜 들어왔으면 다음 입력을 기다린다
    ...
}

```

re , res , rese 까지는 min(cmdlen, 5) 덕분에 입력된 만큼만 "reset" 과 비교되어 일치한다. 즉 "아직 reset이 될 가능성이 있는 입력"으로 살아남는다. 그러다 길이가 5에 도달하는 순간 비로소 실행되고, 만약 중간에 엉뚱한 글자가 끼면 그 시점에서 비교가 깨져 다음 분기로 넘어간다. 이 구조 덕에 미완성 입력과 오류 입력을 자연스럽게 구분할 수 있었다.

led 명령도 같은 원리다.

```

else if (Str_Cmp_N(cmd, "led", min(cmdlen,3)) == 0){
    if( cmdlen < 4 ) break;
    if( *(cmd+3) < '1' || *(cmd+3) > '3' ) { wrongflag=DEF_TRUE; break; }
    lednum = *(cmd+3)-'0';
    ...
}

```

led 까지 맞으면 그 다음 한 글자가 LED 번호여야 한다. '1' ~ '3' 범위를 벗어나면 그건 더 기다려도 정상이 될 리 없으니 곧바로 wrongflag 를 세워 Wrong Command로 보낸다. 번호 뒤로는 on , off , blink t 를 각각 길이 조건과 함께 검사한다. 예를 들어 blink는 ledNblinkT 로 길이가 10이어야 하고, 주기 t 는 '1' ~ '9' 사이여야 한다.

```

else if(Str_Cmp_N(cmd+4, "blink", min(cmdlen-4,5)) == 0){
    if( cmdlen < 10 ) break;
    if( *(cmd+9) < '1' || *(cmd+9) > '9' ) {wrongflag = DEF_TRUE; break;}
    OS_CRITICAL_ENTER();
    LedMode[lednum] = LED_BLINK;
    LedPeriod[lednum] = *(cmd+9) - '0';
    OS_CRITICAL_EXIT();
    UsartPrintLedMsg( lednum, LED_BLINK, *(cmd+9)-'0');
    cmdlen = 0;
}

```

명령이 성공적으로 처리되면 cmdlen 을 0으로 되돌려 buffer를 비운다. 끝까지 어느 분기에도 걸리지 않은 쓰레기 입력은 마지막 else 에서 wrongflag 를 묻고 빠져나간다.

Task 간 공유 변수와 Critical Section

과제의 또 다른 축은 task 간 통신을 global variable로 구현하고 그 접근을 critical section으로 보호하라는 것이었다. LED의 상태와 주기를 담는 전역 배열을 두었다.

```

static ledcmd    LedMode[LED_COUNT+1];
static int       LedPeriod[LED_COUNT+1];
static char      CmdBuffer[CMD_BUF_SIZE];
static int       CmdLen;

```

USART task(정확히는 그것이 호출하는 parser)는 명령에 따라 LedMode[] , LedPeriod[] 를 쓰고, LED task는 주기적으로 이 값을 읽어 실제 LED를 제어한다. 같은 변수를 두 task가 만지니 OS_CRITICAL_ENTER() /

`OS_CRITICAL_EXIT()` 로 감쌌다.

LED task에서 한 가지 신경 쓴 부분이 있다. critical section 안에서는 공유 값을 local 변수로 복사만 하고, 실제 LED를 켜고 끄는 무거운 작업은 critical section 밖에서 한다는 점이다.

```
OS_CRITICAL_ENTER();
mode = LedMode[led];
period = LedPeriod[led];
OS_CRITICAL_EXIT();

if(mode == LED_ON || (mode == LED_BLINK && ledstats[led] == 1u)){
    BSP_LED_On(led);
    ...
}
```

이렇게 한 이유는 critical section을 최대한 짧게 유지하기 위해서다. critical section은 인터럽트를 막아버리니 그 안에서 BSP 호출 같은 시간이 걸리는 일을 하면 시스템 전체의 응답성이 나빠진다. 한 번의 판단에 필요한 `mode` 와 `period` 만 원자적으로 읽어오고, 나머지 처리는 풀어준 상태에서 하는 것이 정석이다. blink는 LED task가 1초마다 도는 주기를 카운트해서, 누적 초가 지정한 period에 도달하면 상태를 반전시키는 방식으로 구현했다.

Interrupt 대신 Task polling을 택한 이유

USART 수신을 interrupt 방식으로 구현할까도 진지하게 고민했다. 하드웨어 관점에서는 그쪽이 더 자연스럽긴 하다. 그러나 ISR에서 문자를 받아버리면 정작 `AppTaskUsart()` 는 할 일이 없어진다. 그러면 과제가 요구한 "USART task와 LED task를 별도로 생성하고 task 간 통신을 구현하라"는 그림이 어색해진다. ISR이 일을 다 해버리는데 USART task를 따로 둘 이유가 빈약해지는 것이다.

게다가 ISR 안에서 문자열 parsing이나 USART 출력 같은 무거운 처리를 하는 건 임베디드에서 피해야 할 패턴이다. ISR은 짧고 빠르게 끝나야 한다. 결국 USART task가 직접 polling으로 입력을 확인하고 parser를 호출하는 구조가 과제 요구사항에도, 구현·디버깅 난이도에도 더 들어맞는다고 판단해 그쪽으로 갔다.

버퍼 오버플로우와 Enter 처리

한 글자씩 buffer에 쌓는 구조에는 함정이 하나 있다. 명령이 완성되지 않은 채 사용자가 계속 글자를 치면 `CmdBuffer` 가 넘칠 수 있다. 그래서 buffer에 넣기 직전에 길이를 검사해, 한계에 닿으면 비우고 Wrong Command로 처리하도록 막았다.

```
OS_CRITICAL_ENTER();
if (CmdLen >= (int)(CMD_BUF_SIZE - 1)) {
    CmdLen = 0;
    OS_CRITICAL_EXIT();
    UsartPrint("\r\nWrong Command\r\n");
    continue;
}
CmdBuffer[CmdLen++] = ch;
CmdBuffer[CmdLen] = '\0';
OS_CRITICAL_EXIT();
```

Enter 처리도 손봤다. 시리얼 터미널에서는 줄바꿈을 위해 `\r\n` 을 함께 보내줘야 다음 줄 맨 앞으로 커서가 가는 경우가 많다. Enter가 들어오면 buffer를 비우고 `\r\n` 을 출력하도록 했다.

```
if (ch == '\r' || ch == '\n') {
    OS_CRITICAL_ENTER();
    CmdLen = 0;
    OS_CRITICAL_EXIT();
    UsartPrint("\r\n");
    continue;
}
```

명령 처리 결과 피드백

명령이 먹혔는지 사용자가 알 수 있도록, 처리에 성공하면 어떤 LED가 어떤 상태가 됐는지 시리얼로 되돌려주는 `UsartPrintLedMsg()` 를 추가했다. printf 같은 표준 출력이 없는 환경이라 한 글자씩 직접 찍는 함수다.

```
static void UsartPrintLedMsg(CPU_INT08U led, ledcmd mode, CPU_INT08U period){
    UsartPrint("\r\nLED ");
    UsartPutChar('0' + led);
    UsartPrint(" ");

    if(mode == LED_ON) UsartPrint("On");
    else if(mode == LED_OFF) UsartPrint("Off");
    else if(mode == LED_BLINK) UsartPrint("Blink");

    if (period > 0u) {
        UsartPrint(" ");
        UsartPutChar('0' + period);
    }

    UsartPrint("\r\n");
}
```

가령 `led1blink3` 을 치면 `LED 1 Blink 3` 이 출력되고, `reset` 을 치면 세 LED에 대해 차례로 `LED 1 Off` , `LED 2 Off` , `LED 3 Off` 가 찍힌다. 눈으로 LED만 보는 것보다 훨씬 확실한 피드백이다.

최종 동작

최종적으로 구현한 `app.c` 는 다음과 같이 동작한다.

입력	결과
<code>led1on</code>	LED1 켜짐
<code>led2off</code>	LED2 꺼짐
<code>led3blink1</code>	LED3이 1초 주기로 blink
<code>led1blink3</code>	LED1이 3초 주기로 blink
<code>reset</code>	LED1, LED2, LED3 모두 꺼짐
잘못된 명령	<code>Wrong Command</code> 출력 후 buffer 초기화

빌드는 다음 명령으로 확인했고, 정상적으로 ELF가 생성됐다.

```
cmake --build build/ninja
```

마치며

이번 과제는 단순히 LED를 켜고 끄는 일이 아니었다. RTOS 위에서 task를 분리하고, 그 사이를 잇는 shared variable을 critical section으로 안전하게 다루는 것이 본질이었다. 특히 USART 입력이 한 글자씩 들어온다는 사실 하나 때문에, "아직 완성되지 않은 명령"과 "틀린 명령"을 구분하는 parser 로직을 고민해야 했던 점이 인상 깊었다. `Str_Cmp_N` 의 비교 길이를 `min` 으로 잘라내는 작은 트릭 하나로 이 문제가 깔끔하게 풀린 것이 특히 만족스러웠다.

정리하면, USART task는 입력 수신과 command parsing을 맡고, LED task는 global command state를 읽어 LED를 제어한다. 이로써 과제가 요구한 USART echo, LED blink, LED command control, 그리고 critical section 적용까지 모두 충족했다. 개발 환경을 N100 미니 PC로 옮긴 덕에 어디서든 붙어서 작업할 수 있었던 것은 덤이었다.